

DS210 Final Project Written Report

Name: Ranbir Jain

A. Project Overview:

- Goal: The goal of my final project for this class is to analyze a graph of stanford.edu pages (where nodes represent pages and edges represent hyperlinks between pages). Specifically, my project analyzes the graph to know if it follows a power-law distribution, which can be helpful in understanding the structure of the graph and if it behaves like other real-world large networks (most large networks follow the power-law distribution, where a few pages (“hub” pages) have high degrees). Certain algorithmic operations are more efficient on the power-law distribution, and nodes with high degrees may indicate pages of more importance. If this graph doesn’t follow a power-law distribution, it may indicate an anomaly → the stanford.edu graph might not be structured like other real-world large networks.

- Dataset: The source for my dataset is the “Stanford web graph” page on the SNAP website (Stanford Network Analysis Project), specifically linked at this [site](#). The size of the dataset is roughly 32.9 megabytes (MB), consisting of over 280000+ nodes and 2.3+ million edges.

B. Data Processing:

- Data Loading, Cleaning, and Transformations: First, I downloaded the data from the link (SNAP website). The file I downloaded was “web-Stanford.txt.gz”. I needed the graph to be in a .txt format, so I started with a gunzip decompression to handle the .gz GNU zip compression. Once I obtained the .txt file, I created a new Rust project for my final project and moved the file to the project folder. Upon inspecting the .txt file to understand the nature of my data, I realized that the first few lines of the data had text information (those lines started with a hashtag) that I did not want to parse (I only wanted to parse the nodes and edges), so I skipped those lines using a .startswith() expression in my graph parsing function in the graph module. The only transformation that was needed was a log transformation of certain values (degrees and counts) before plotting. Overall, very minimal data cleaning and transformation was required.

C. Code Structure:

- Modules: I wrote 5 modules that I used in my main.rs file. These modules were:
 1. **Graph**: The graph module contains the `parse_graph` function that takes in a graph file and turns it into an adjacency list format.
 2. **Degree**: The degree module contains the `compute_degree_distribution` function used for computing the in-degree and out-degree distributions, stored together in a `DegreeDistribution` custom datatype (created with a struct).
 3. **Regression**: The regression module contains the `lin_regression` function to perform a linear regression using two sets of points (degree and counts in the scope of this project), with an additional function for computing the r-squared value for fit of a regression.
 4. **Transform**: The transform module contains the `log_transform` function to transform two sets of points into natural log values.
 5. **Plotting**: The plotting module contains the `log_log_plot` function that uses two sets of points along with the slope and intercept to create a log-log plot with a best line of fit and returns it to a filename of choosing passed in the argument.

The rationale behind splitting most of my code into modules was because I wanted to separate each important task into a separate section. I completed my data cleaning and graph parsing in the “Graph” module, isolating File I/O tasks. The “Degree” module was separated as it computes degree distribution tasks, essentially keeping the structural logic for the graph in one place (computing degree distribution and sorting that distribution). The “Regression” module covers statistical modeling and fitting, with linear regression computations and metrics for the regression (all statistical-based logic kept together). The “Transform” module transforms the data by applying the log transforms, separated from the other modules as this module covers pre-processing mathematical logic. Finally, the “Plotting” module was separated because while it is related to the “Transform” and “Regression” modules, I wanted to separate the visualizations / output product from all the internal processes.

- Key Functions and Types:

—
1. `graph::parse_graph`

Purpose:

- Parse a .txt file in SNAP edge-list format into an adjacency list.

Inputs:

- ``file_path: &str`` – path to a graph data file.

Outputs:

- ``Vec<Vec<usize>>`` – adjacency list where index is node ID and the value is the list of edges / neighbors.

Core Logic and Key Components:

- Reads the file line-by-line and skips commented and empty lines.
- Splits lines into parts and parses them into ``from`` and ``to`` node IDs.
- Tracks the node ID to come up with the size of the adjacency list.
- Returns the graph as a ``Vec<Vec<usize>>`` adjacency list.

—

2. `degree::compute_degree_distribution`

Purpose:

- Computes in-degree and out-degree distributions for a directed graph.

Inputs:

- ``&Vec<Vec<usize>>`` – adjacency list representation of a graph

Outputs:

- ``DegreeDistributions`` – a struct consisting of two fields called `in_degrees` and `out_degrees`, both containing `HashMap<usize, usize>`.

Core Logic and Key Components:

- Iterate through the neighbors of each node to count out-degrees.
- Increment the `in_degrees` counter for each neighbor.
- Iterates over `in_degrees` vector to increment counts in `in_degree_count`.
- Wrap the HashMaps into a `DegreeDistributions` struct and return.

—

3. `degree::sort_degree_distribution`

Purpose:

- Sorts a degree distribution by a particular sorting criteria.

Inputs:

- ``&HashMap<usize, usize>`` – HashMap representing a single degree distribution.
- ``SortCriterion`` – an enum consisting of two sorting criterion options (`ByCount`

and ByDegree).

Outputs:

- ``Vec<(usize, usize)>`` – vector sorted by sorting criterion with (degree, count) pairs returned.

Core Logic and Key Components:

- Collect key-value pairs into a vector.
- Match by selected criterion for the corresponding logic.
- Sort by selected criterion from highest to lowest using ``std::cmp::Reverse``.
- Turn key-value pairs into tuples and return the sorted vector.

—

4. `transform::log_transform`

Purpose:

- Apply natural logarithm transform to a degree distribution for log plotting.

Inputs:

- ``&HashMap<usize, usize>`` – HashMap representing degree distribution.

Outputs:

- ``(Vec<f64>, Vec<f64>)`` – tuple of f64 vectors consisting of $\ln(\text{degree})$ and $\ln(\text{count})$.

Core Logic and Key Components:

- Create two vectors for storing natural log values.
- Iterate through degree distribution and filter out zero degrees or zero counts.
- Converts degree and counts to f64, take natural log, and push to vectors.
- Return $\ln(\text{degree})$ and $\ln(\text{count})$ vectors in a tuple.

—

5. `regression::lin_regression`

Purpose:

- Fit a simple linear regression to log-log degree distribution.

Inputs:

- ``&[f64], &[f64]`` – x and y values – $\ln(\text{degrees})$ and $\ln(\text{counts})$.

Outputs:

- `Option<(f64, f64)>` – (slope, intercept) of best fit line.

Core Logic and Key Components:

- Compute means of x and y.
- Apply least-squares regression formulas.
- Return None if there's any mismatch, else return calculated slope and intercept.

—

6. regression::r_squared

Purpose:

- Compute the r^2 value (goodness of fit) for the regression model.

Inputs:

- `x: &[f64], y: &[f64]` – x and y values – ln(degrees) and ln(counts).
- `slope: f64` – slope of line
- `intercept: f64` – intercept of line

Outputs:

- `Option<f64>` – r^2 value between 0 and 1

Core Logic and Key Components:

- Compute SSE (sum of squared errors) and SST (sum of squared total)
- Return $1 - (SSE/SST)$ as r^2 value (or none if $SST = 0$).

—

7. plotting::log_log_plot

Purpose:

- Generate a PNG plot of the log-log degree distribution and best fit line.

Inputs:

- `filename: &str` – name of file the generated PNG plot will be stored on.
- `x: &[f64], y: &[f64]` – ln(degrees) and ln(counts) slices.
- `slope: f64` – slope of line
- `intercept: f64` – intercept of line

Outputs:

- `Result<()>, Box<dyn std::error::Error>>` – returns nothing if successful, else none if there's an error.

Core Logic and Key Components:

- Use `plotters` crate to render the plot with axes and points.
- Draw best-fit regression line
- Save output as a PNG file

—

8: degree::DegreeDistributions (struct)

Purpose:

- Store the in and out degree distributions of a directed graph as HashMaps.

Inputs:

- Constructed from compute_degree_distribution

Outputs:

- Used as a return type / data type that encapsulates both distributions.

Core Logic and Key Components:

- Holds two public fields that consist of HashMap<usize, usize>
- One field for in-degree distributions – degrees and counts.
- One field for out-degree distributions – degrees and counts.
- Modular access through .in_degrees and .out_degrees.

—

9: degree::SortCriterion (enum)

Purpose:

- Specify the criterion for sorting a distribution – by count or by degree.

Inputs:

- Passed to sort_degree_distribution() to select sorting behavior.

Outputs:

- Determines ordering logic for returned tuple of degrees and counts.

Core Logic and Key Components:

- Selector for sorting behavior.
- Works through use of `match` statement in sorting function to provide correct sorting functionality.

—

- Main Workflow: The main objective of the project is to analyze the in and out-degree distributions of the Stanford web graph, check for power-law behavior, and visualize log-log plots with regression for further analysis and evidence.

The first step is to parse the graph using the ``parse_graph`` function in the graph module. After reading the ``web-Stanford.txt`` graph, the next step is to clean the data by removing lines that are comments or empty. Once that's completed, the lines are read and split into two parts (the "from" node and the "to" node). These are subsequently stored in a vector to store all graph edges. Another vector is allocated with size based on tracking of the max vertex. Iterating through the graph edges vectors and pushing the "to" values to the "from" index of the vector leaves us with an adjacency list representation of the graph returned in the form of `Vec<Vec<usize>>`.

The next step is the degree distribution computation, where the graph is parsed into the ``compute_degree_distribution`` function, after which the in and out-degrees and their counts are computed and stored into a `DegreeDistributions` datatype (this datatype stores two `HashMap`s, one for in-degree distribution and one for out-degree distribution). After this, the top 5 degrees and counts are printed using the `sort_degree_distribution` function, which takes in one of the distributions (`&HashMap<usize, usize>`) and uses matching on an enum called `SortCriterion` (which determines how the distribution should be sorted) to sort the distribution by highest to lowest by that criteria. We first sort by counts and print before sorting by degrees and printing (in `main.rs`).

Once the degree distributions are computed and sorting is completed, the next part is the log transformation, where both the in and out-degree distributions are parsed into `log_transform` to obtain the natural log values of degrees and counts for both distributions. We need $\ln(\text{degrees})$ and $\ln(\text{counts})$ given the nature of the power-law distributions as non-linear, which is only linearized when we take the log. Only after we linearized the power-law distribution (by taking $\ln(\text{degrees})$ and $\ln(\text{counts})$) would it be appropriate to use a linear regression model.

After we have the log transformed values, they are parsed into the `lin_regression` function to perform a linear regression, from which we obtain the slope and intercept. We also pass in those log transformed values along with the slope and intercept to calculate the r-squared values for the distributions (which is calculated through the `r_squared` function which calculates SSE (sum of squared errors) and SST (sum of squared totals) before calculating $1 - (\text{SSE} / \text{SST})$ to give the final output. The power-law exponent estimates are also calculated by simply taking the negative of the slope (given that slope itself is the negative of alpha (where alpha is the exponent

estimate) because of the form of the straight line after taking the log transform where the coefficient of the covariate $\ln(\text{degree})$ is negative alpha). All these metrics (slope, intercept, power-law exponent estimate, and r-squared value) are printed for both distributions.

The final step is the log-log plot, which is created through the `log_log_plot` function in the plotting module. The $\ln(\text{degrees})$, $\ln(\text{counts})$, slope, and intercept values are parsed in along with a string reference for a filename on which the final plot is visualized and stored. Using the `plotters` crate, a scatterplot is created (with points colored in blue) and a best line of fit (colored in red). This function is used twice to create two different PNGs (“`in_degree_fit.png`” and “`out_degree_fit.png`”) for the two distributions, which show up in the project folder after running ``cargo run --release``.

D. Tests:

• ``cargo test`` output: The output from ``cargo test`` is as follows:

```
(base) ran@crc-dot1x-nat-10-239-105-120 ds210finalproj % cargo test
Compiling ds210finalproj v0.1.0 (/Users/ran/Desktop/ds210/ds210finalproj)
Finished `test` profile [unoptimized + debuginfo] target(s) in 1.67s
Running unittests src/main.rs (target/debug/deps/ds210finalproj-0398ea1b416a61e3)

running 3 tests
test test::test_lin_regression ... ok
test test::test_comp_degree_distribution ... ok
test test::test_log_transform ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

All 3 tests show up with a test result of “passed”.

• Use of tests:

The first test is the `test_comp_degree_distribution` test, which ensures that the `compute_degree_distribution` function correctly calculates the distributions by passing in a small hardcoded graph and checking whether the in and out-degrees are correctly counted. This test matters because it ensures that the graph metrics are correct; any mistake here would mean that all the analysis and regression of the graph would be invalid.

The second test is the `test_log_transform` test, which confirms that the `log_transform` function works correctly by filtering out the zeroes and applying the natural log. This is done through inputs of key-value pairs and making sure only 1 of those pairs should remain (greater than 0) to be subsequently transformed. This step is important because

if the values aren't log transformed, then the linear regression and model fit will be completely meaningless for our analysis.

The third test is the `test_lin_regression` test, which makes sure that `lin_regression` and `r_squared` return roughly the correct values. This is done by creating two sets of vectors to indicate the points based on a formula of $y = 3x + 1$, after which it is checked whether the slope is roughly 3.0, the intercept is 1.0, and the r-squared value is 1.0. This test matters to ensure `lin_regression` and `r_squared` are working correctly because these are important for estimating power-law behavior, and ensures that the slope and intercept calculations are precise.

E. Results:

The results of my project are the following:

```
281904 nodes in graph.
```

```
Top in-degree counts:
Degree 2: 69915 nodes
Degree 1: 66816 nodes
Degree 3: 40381 nodes
Degree 4: 21052 nodes
Degree 0: 20316 nodes
```

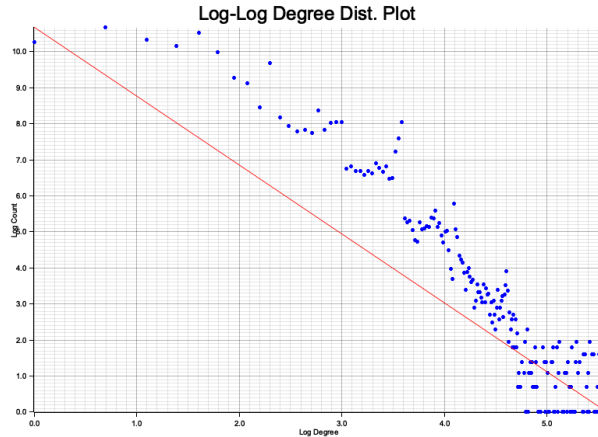
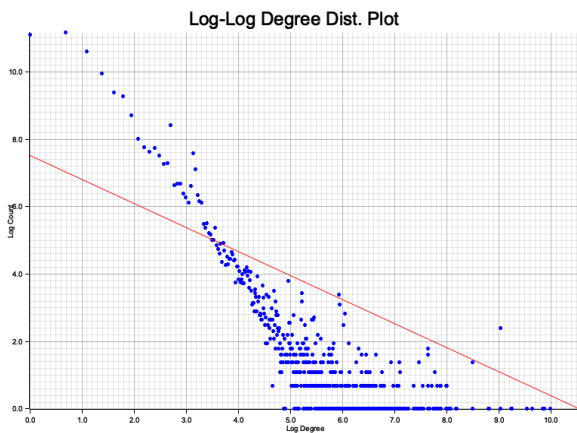
```
Top out-degree counts:
Degree 2: 42916 nodes
Degree 5: 37693 nodes
Degree 3: 31033 nodes
Degree 1: 28634 nodes
Degree 4: 26075 nodes
```

```
Top in-degree Degrees:
Degree 38606: 1 nodes
Degree 21920: 1 nodes
Degree 19457: 1 nodes
Degree 19377: 1 nodes
Degree 19003: 1 nodes
```

```
Top out-degree Degrees:
Degree 255: 1 nodes
Degree 247: 5 nodes
Degree 245: 2 nodes
Degree 244: 2 nodes
Degree 243: 2 nodes
```

```
In-degree distribution Metrics:
Slope: -1.0587, Intercept: 7.5214
Power-law exponent estimate: 1.0587
R^2 (r-squared) value: 0.6289
```

```
Out-degree distribution Metrics:
Slope: -2.6624, Intercept: 14.8287
Power-law exponent estimate: 2.6624
R^2 (r-squared) value: 0.8974
```



(Left screenshot shows the log-log “in-degree” distribution plot, right screenshot shows the log-log “out-degree” distribution plot)

The first screenshot covers several metrics, starting with the number of nodes in the graph (281904 nodes). This is followed by the top 5 in-degree and out-degree counts by count descending, in which the degrees itself are quite low (generally between degree 0 – 5). On the in-degree side, Degree 2 has 69915 nodes, indicating that 69915 nodes / pages have 2 incoming links. On the out-degree side, Degree 5 has 37693 nodes, indicating that 37693 nodes / pages link to 5 other nodes / pages. Overall, such behavior is showing that most nodes are lightly connected, consistent with the behavior of real-world networks. After this, the top 5 largest in-degree and out-degree Degrees are calculated, often with a few number of nodes (count). On the in-degree side, Degree 38606 has 1 node for instance, indicating that node has 38,606 incoming links. Meanwhile, on the out-degree side, Degree 255 has 1 node, indicating that 1 node is linking to 255 other nodes. Such results indicate the presence of high-degree “hub” pages, which also follows the behavior of large networks. In the context of stanford.edu webpages, this would make sense as certain stanford.edu pages (like the homepage, admissions page, and so on) may have more activity with relation to links.

These metrics are further explored through the calculation of slope, intercept, power-law exponent estimate, and r-squared value on the natural log of the degrees and counts. For the in-degree, the exponent estimate is ~ 1.0587 and the r-squared is around 0.6289, which indicates that the in-degree distribution seems to loosely follow a power-law pattern as the fit is imperfect. There may be deviations with relation to stanford.edu pages based on the structure and linking of the pages, but this may not necessarily be uncommon. This can be visually seen in the screenshot of the graph on the left, where the best fit line isn’t perfect, but isn’t super far from most of the values). For the out-degree, the exponent estimate is ~ 2.66 and the r-squared value is 0.8974, which indicates that the out-degree distribution shows a stronger fit to a power-law

distribution, suggesting a small number of pages are highly connected, while most link to a few others only. These results are further supported by the graph on the right side, where the best fit line is much closely tracked to the points.

Overall, many nodes with low degree, very few nodes with high degree, and reasonably power-law with respect to fit (especially for the out-degree distribution).

—

F. Usage Instructions:

To compile the project, use ``cargo build``. To run the executable, use ``cargo run --release``. Running ``cargo run --release`` will parse the `.txt` dataset into a graph (as long as `web-Stanford.txt` is within the root folder of the project directory), then compute the in and out-degree distributions, perform the log transformations and linear regression, print analysis results and metrics to the terminal, and output two PNG plots. The project doesn't need any other arguments. PNG filenames are currently hardcoded, specifically because there are only two plots (in-degree and out-degree plot). Those filenames can be changed in `main.rs`. The project runtime should be between 2 - 5 seconds under `--release` optimized run.

—

G. AI Assistance Disclosure and Other Citations:

A substantive portion where I used ChatGPT was for understanding how I could figure out if my data follows a power-law distribution. (Screenshot below).

How do I know if my data follows a power-law distribution?

To assess whether your data follows a **power-law distribution**, you can evaluate it using several key principles:

1. Look at the Shape on Log-Log Axes

- Plot the data on **log-log scales**—for example, using a histogram or a complementary cumulative distribution function (CCDF).
- A **straight-line pattern** in the **upper tail** suggests a power-law relationship.

2. Identify the Tail Behavior

- Power-law behavior typically holds **only above a certain minimum value** (often denoted $x_{\min}$).
- Focus your analysis on values **greater than $x_{\min}$** —this is where the power-law form is expected to dominate.

3. Estimate the Slope

- The slope of the line on a log-log plot corresponds to the **exponent α** of the power-law.
- This exponent is key to confirming the nature of the distribution (typical values are between 2 and 3 in real-world systems).

4. Compare Alternative Fits

- Check if **log-normal**, **exponential**, or **stretched exponential** models fit the data as well or better.
- A visual fit alone is **not sufficient**—these distributions can look similar on log-log plots.

5. Evaluate Statistical Fit

- Use a **goodness-of-fit test** such as the Kolmogorov–Smirnov (KS) test to compare your data to a fitted power-law.
- You can also compute a **p-value** via resampling: a high p-value (e.g., > 0.1) suggests the power-law is plausible.

6. Be Skeptical of Small Samples

- Power-law detection is unreliable on **small datasets**.
- You need enough observations in the tail for a reliable conclusion.

Would you like to walk through a real or hypothetical dataset to practice applying these checks?

🗨️ 📄 🗑️ 🔄 📌 📁

This screenshot shows that I asked GPT how to determine if my data follows a power-law distribution (as I had never explored graphs in relation to the power-law distribution and needed to know what benchmark / metric to test on). From this answer, I understood that I needed to analyze the log-log plot because of the straight line pattern needed, which further told me that was due to the nature of the distribution (further corroborated by additional source I read: <https://www.statisticshowto.com/power-law/>). To do this, I understood I would need to estimate the slope (which was suggested in #3) to create the best fit of line (suggested in #5 – while it doesn't particularly state best fit line, I decided to go ahead with that since it's part of evaluating statistical fit + it's mentioned in the suggestion for #1). Without such a prompt, I wouldn't have understood that I need the log transformed values because the power-law distribution hasn't been linearized. If I didn't transform the values, then my linear regression, r-squared values, and visualizations in plotting all would have been incorrect.

Another time I had used GPT was for understanding in-degree and out-degree distributions when coming up with my project proposal, but I haven't cited that GPT

usage because that was part of the common knowledge for coming up with my proposal based on the Piazza post of projects we could work on. However, I did ask GPT about how I would go about storing the in-degree and out-degree distributions:

How could I store in-degree and out-degree distributions?

To store **in-degree** and **out-degree** distributions efficiently, you want a data structure that counts **how many nodes** have each degree. A good choice in most languages (including Rust, Python, etc.) is a:

👉 **HashMap / Dictionary:**

- **Key** = degree (e.g., 0, 1, 2, ...)
- **Value** = number of nodes with that degree

Example:

```
text 📄 Copy ✎ Edit

In-degree distribution:
0 → 20316 nodes
1 → 66816 nodes
2 → 69915 nodes
...

Out-degree distribution:
0 → 10000 nodes
1 → 28634 nodes
2 → 42916 nodes
...
```

This lets you:

- Quickly update counts as you process the graph
- Easily query or sort degrees by frequency

From using GPT, I learned and understood that a HashMap / dictionary would be the best data structure to use for storing the in-degree and out-degree distributions because we want to have information about the number of edges connecting to the node (degree) and the number of nodes that have that degree (count). This is especially helpful for sorting, as one could store by degree or by count, which helped come up with the idea behind using an enum for the sorting criteria. In addition, by having the degree and counts as HashMaps, I understood I would have two sets of points that could be analyzed further with regression, helping me get closer towards analyzing whether the graph follows power-law.

Other sources I used include the lecture notes for the class. For instance, lecture 21 goes over the use of the plotters crate and creating plots. I had to heavily rely on this lecture to come up with my `log_log_plot` function and the regression plots I subsequently created since I did not have much hands-on experience working with plotting in Rust before. One more source was one of the homeworks (HW #6 and #7 to be particular), as my graph function is heavily derived from the graph reading / parsing functions I made for these homeworks (specifically because the implementation of the graph as an adjacency list representation lends itself well to this case). Apart from these, suggestions from the compiler and Rust documentation were utilized for certain changes (such as adding `.sum::<f64>()` as part of `mean_x` and `mean_y` computations in

lin_regression and for use of methods). Finally, for certain calculations in lin_regression and r_squared (especially SSE and SST), I used information from previous statistics classes like MA416 for those calculations as I manually had to calculate these metrics (slope, intercept, SSE, SST, r-squared, etc) in that class.